

Lecture 6

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

What we'll do:

- cost analysis of basic operations
- identify basic solution schemes to systems
- Solving Triangular Systems

What's the problem?

- humans: milliFLOPS
- hand calculators: 10 FLOPS
- desktops: a few GFLOPS (10^9 FLOPS)

look at the basic operations!

BLAS

Basic Linear Algebra Subprograms (BLAS) interface introduced APIs for common linear algebra tasks

- Level 1: vector operations (dot products, vector norms, etc) e.g.

$$y \leftarrow \alpha x + y$$

- Level 2: matrix-vector operations, e.g.

$$y \leftarrow \alpha Ax + By$$

- Level 3: matrix-matrix operations, e.g.

$$C \leftarrow \alpha AB + \beta C$$

- optimized versions of the reference BLAS are used everyday: ATLAS, etc.

vec-vec, mat-vec, mat-mat

- inner product of u and v both $[n \times 1]$

$$\sigma = u^T v = u_1 v_1 + \cdots + u_n v_n$$

- $\rightarrow n$ multiplies, $n - 1$ additions
- $\rightarrow \mathcal{O}(n)$ flops

vec-vec, mat-vec, mat-mat

- mat-vec of A ($[n \times n]$) and u ($[n \times 1]$)

```
1   for  $i = 1, \dots, n$   
2       for  $j = 1, \dots, n$   
3            $v(i) = a(i, j)u(j) + v(i)$   
4       end  
5   end
```

- $\rightarrow n^2$ multiplies, n^2 additions
- $\rightarrow \mathcal{O}(n^2)$ flops

vec-vec, mat-vec, mat-mat

- mat-mat of A ($[n \times n]$) and B ($[n \times n]$)

```
1   for  $j = 1, \dots, n$ 
2       for  $i = 1, \dots, n$ 
3           for  $k = 1, \dots, n$ 
4                $C(k, j) = A(k, i)B(i, j) + C(k, j)$ 
5           end
6       end
7   end
```

- $\rightarrow n^3$ multiplies, n^3 additions
- $\rightarrow \mathcal{O}(n^3)$ flops

vec-vec, mat-vec, mat-mat

Operation	FLOPS
$u^T v$	$\mathcal{O}(n)$
Au	$\mathcal{O}(n^2)$
AB	$\mathcal{O}(n^3)$

matlab test

four tests:

- matrix-matrix multiply
- inner product
- matrix-vector multiply
- n matrix-vector multiplies

order them...fastest to slowest

Gaussian Elimination

- Solving Diagonal Systems
- Solving Triangular Systems
- Gaussian Elimination Without Pivoting
 - Hand Calculations
 - Cartoon Version
 - The Algorithm
- Gaussian Elimination with Pivoting
 - Row or Column Interchanges, or Both
 - Implementation
- Solving Systems with the Backslash Operator

Solving Diagonal Systems

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

Solving Diagonal Systems

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

is equivalent to

$$\begin{aligned} x_1 &= -1 \\ 3x_2 &= 6 \\ 5x_3 &= -15 \end{aligned}$$

Solving Diagonal Systems

The system defined by

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 5 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ 6 \\ -15 \end{bmatrix}$$

is equivalent to

$$\begin{aligned} x_1 &= -1 \\ 3x_2 &= 6 \\ 5x_3 &= -15 \end{aligned}$$

The solution is

$$x_1 = -1 \qquad x_2 = \frac{6}{3} = 2 \qquad x_3 = \frac{-15}{5} = -3$$

Solving Diagonal Systems

Listing 1: Diagonal System Solution

```
1  given  $A, b$   
2  for  $i = 1 \dots n$   
3       $x_i = b_i / a_{i,i}$   
4  end
```

In Matlab:

```
1 >> A = ...           % A is a diagonal matrix  
2 >> b = ...  
3 >> x = b ./ diag(A)
```

This is the *only* place where element-by-element division (`./`) has anything to do with solving linear systems of equations.

Operations?

Try...

Sketch out an operation count to solve a diagonal system of equations...

Operations?

Try...

Sketch out an operation count to solve a diagonal system of equations...

cheap!

one division n times $\longrightarrow \mathcal{O}(n)$ FLOPS

Triangular Systems

The generic lower and upper triangular matrices are

$$L = \begin{bmatrix} l_{11} & 0 & \cdots & 0 \\ l_{21} & l_{22} & & 0 \\ \vdots & & \ddots & \vdots \\ l_{n1} & & \cdots & l_{nn} \end{bmatrix}$$

and

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & & u_{2n} \\ \vdots & & \ddots & \vdots \\ 0 & & \cdots & u_{nn} \end{bmatrix}$$

The triangular systems

$$Ly = b \qquad Ux = c$$

are easily solved by **forward substitution** and **backward substitution**, respectively

Solving Triangular Systems

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

Solving Triangular Systems

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

is equivalent to

$$\begin{array}{rclcl} -2x_1 & + & x_2 & + & 2x_3 & = & 9 \\ & & 3x_2 & + & -2x_3 & = & -1 \\ & & & & 4x_3 & = & 8 \end{array}$$

Solving Triangular Systems

$$A = \begin{bmatrix} -2 & 1 & 2 \\ 0 & 3 & -2 \\ 0 & 0 & 4 \end{bmatrix} \quad b = \begin{bmatrix} 9 \\ -1 \\ 8 \end{bmatrix}$$

is equivalent to

$$\begin{array}{rclcl} -2x_1 & + & x_2 & + & 2x_3 & = & 9 \\ & & 3x_2 & + & -2x_3 & = & -1 \\ & & & & 4x_3 & = & 8 \end{array}$$

Solve in backward order (last equation is solved first)

$$x_3 = \frac{8}{4} = 2$$

$$x_2 = \frac{1}{3}(-1 + 2x_3) = \frac{3}{3} = 1$$

$$x_1 = \frac{1}{-2}(9 - x_2 - 2x_3) = \frac{4}{-2} = -2$$

Solving Triangular Systems

Solving for $x_1, x_2, \dots, x_n$ for a lower triangular system is called **forward substitution**.

```
1  given  $L, b$ 
2   $x_1 = b_1/\ell_{11}$ 
3  for  $i = 2 \dots n$ 
4       $s = b_i$ 
5      for  $j = 1 \dots i - 1$ 
6           $s = s - \ell_{i,j}x_j$ 
7      end
8       $x_i = s/\ell_{i,i}$ 
9  end
```

Solving Triangular Systems

Solving for $x_1, x_2, \dots, x_n$ for a lower triangular system is called **forward substitution**.

```
1  given  $L, b$ 
2   $x_1 = b_1/\ell_{11}$ 
3  for  $i = 2 \dots n$ 
4       $s = b_i$ 
5      for  $j = 1 \dots i - 1$ 
6           $s = s - \ell_{i,j}x_j$ 
7      end
8       $x_i = s/\ell_{i,i}$ 
9  end
```

Using forward or backward substitution is sometimes referred to as performing a **triangular solve**.

Operations?

Try...

Sketch out an operation count to solve a triangular system of equations...

Operations?

Try...

Sketch out an operation count to solve a triangular system of equations...

cheap!

- begin in the bottom corner: 1 div
- row -2 : 1 mult, 1 add, 1 div, or 3 FLOPS
- row -3 : 2 mult, 2 add, 1 div, or 5 FLOPS
- row -4 : 3 mult, 3 add, 1 div, or 7 FLOPS
- $\vdots$
- row $-j$: about $2j$ FLOPS

Total FLOPS? $\sum_{j=1}^n 2j = 2\frac{n(n+1)}{2}$ or $\mathcal{O}(n^2)$ FLOPS

Gaussian Elimination

Gaussian elimination is a mostly general method for solving square systems.

We will work with systems in their matrix form, such as

$$x_1 + 3x_2 + 5x_3 = 4$$

$$9x_1 + 7x_2 + 8x_3 = 6$$

$$3x_1 + 2x_2 + 7x_3 = 1,$$

in its equivalent matrix form,

$$\begin{bmatrix} 1 & 3 & 5 \\ 9 & 7 & 8 \\ 3 & 2 & 7 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 4 \\ 6 \\ 1 \end{bmatrix}.$$

Gaussian Elimination

- Triangular systems are easy to solve in $\mathcal{O}(n^2)$ FLOPS
- Goal is to transform an arbitrary, square system into an equivalent upper triangular system
- Then easily solve with backward substitution

This process is equivalent to the *formal solution* of $Ax = b$, where A is an $n \times n$ matrix.

$$x = A^{-1}b$$

In MATLAB:

```
1 >> A = ...  
2 >> b = ...  
3 >> x = A\b
```

Gaussian Elimination — Hand Calculations

Solve

$$x_1 + 3x_2 = 5$$

$$2x_1 + 4x_2 = 6$$

Subtract 2 times the first equation from the second equation

$$x_1 + 3x_2 = 5$$

$$-2x_2 = -4$$

This equation is now in triangular form, and can be solved by backward substitution.

Gaussian Elimination — Hand Calculations

The elimination phase transforms the matrix and right hand side to an equivalent system

$$\begin{array}{rcl} x_1 + 3x_2 = 5 & \longrightarrow & x_1 + 3x_2 = 5 \\ 2x_1 + 4x_2 = 6 & & -2x_2 = -4 \end{array}$$

The two systems have the same solution. The right hand system is upper triangular.

Solve the second equation for x_2

$$x_2 = \frac{-4}{-2} = 2$$

Substitute the newly found value of x_2 into the first equation and solve for x_1 .

$$x_1 = 5 - (3)(2) = -1$$

Gaussian Elimination — Hand Calculations

When performing Gaussian Elimination by hand, we can avoid copying the x_i by using a shorthand notation.

For example, to solve:

$$A = \begin{bmatrix} -3 & 2 & -1 \\ 6 & -6 & 7 \\ 3 & -4 & 4 \end{bmatrix} \quad b = \begin{bmatrix} -1 \\ -7 \\ -6 \end{bmatrix}$$

Form the *augmented* system

$$\tilde{A} = [A \ b] = \left[\begin{array}{ccc|c} -3 & 2 & -1 & -1 \\ 6 & -6 & 7 & -7 \\ 3 & -4 & 4 & -6 \end{array} \right]$$

The vertical bar inside the augmented matrix is just a reminder that the last column is the b vector.

Gaussian Elimination — Hand Calculations

Add 2 times row 1 to row 2, and add (1 times) row 1 to row 3

$$\tilde{A}_{(1)} = \left[\begin{array}{ccc|c} -3 & 2 & -1 & -1 \\ 0 & -2 & 5 & -9 \\ 0 & -2 & 3 & -7 \end{array} \right]$$

Subtract (1 times) row 2 from row 3

$$\tilde{A}_{(2)} = \left[\begin{array}{ccc|c} -3 & 2 & -1 & -1 \\ 0 & -2 & 5 & -9 \\ 0 & 0 & -2 & 2 \end{array} \right]$$

Gaussian Elimination — Hand Calculations

The transformed system is now in upper triangular form

$$\tilde{A}_{(2)} = \left[\begin{array}{ccc|c} -3 & 2 & -1 & -1 \\ 0 & -2 & 5 & -9 \\ 0 & 0 & -2 & 2 \end{array} \right]$$

Solve by back substitution to get

$$x_3 = \frac{2}{-2} = -1$$

$$x_2 = \frac{1}{-2} (-9 - 5x_3) = 2$$

$$x_1 = \frac{1}{-3} (-1 - 2x_2 + x_3) = 2$$

Gaussian Elimination — Cartoon Version

Start with the augmented system

$$\begin{bmatrix} x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix}$$

The x 's represent numbers, they are generally *not* the same values.

Begin elimination using the first row as the *pivot row* and the first element of the first row as the pivot element

$$\begin{bmatrix} \boxed{x} & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix}$$

Gaussian Elimination — Cartoon Version

- Eliminate elements under the pivot element in the first column.
- x' indicates a value that has been changed once.

$$\begin{bmatrix} \boxed{x} & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix} \longrightarrow \begin{bmatrix} \boxed{x} & x & x & x & x \\ 0 & x' & x' & x' & x' \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} \boxed{x} & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \\ x & x & x & x & x \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} \boxed{x} & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \end{bmatrix}$$

Gaussian Elimination — Cartoon Version

- The pivot element is now the diagonal element in the second row.
- Eliminate elements under the pivot element in the second column.
- x'' indicates a value that has been changed twice.

$$\begin{bmatrix} x & x & x & x & x \\ 0 & \boxed{x'} & x' & x' & x' \\ 0 & x' & x' & x' & x' \\ 0 & x' & x' & x' & x' \end{bmatrix} \longrightarrow \begin{bmatrix} x & x & x & x & x \\ 0 & \boxed{x'} & x' & x' & x' \\ 0 & 0 & x'' & x'' & x'' \\ 0 & x' & x' & x' & x' \end{bmatrix}$$

$$\longrightarrow \begin{bmatrix} x & x & x & x & x \\ 0 & \boxed{x'} & x' & x' & x' \\ 0 & 0 & x'' & x'' & x'' \\ 0 & 0 & x'' & x'' & x'' \end{bmatrix}$$

Gaussian Elimination — Cartoon Version

- The pivot element is now the diagonal element in the third row.
- Eliminate elements under the pivot element in the third column.
- x''' indicates a value that has been changed three times.

$$\begin{bmatrix} x & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & 0 & \boxed{x''} & x'' & x'' \\ 0 & 0 & x'' & x'' & x'' \end{bmatrix} \longrightarrow \begin{bmatrix} x & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & 0 & \boxed{x''} & x'' & x'' \\ 0 & 0 & 0 & x''' & x''' \end{bmatrix}$$

Gaussian Elimination — Cartoon Version

Summary

- Gaussian Elimination is an orderly process for transforming an augmented matrix into an equivalent upper triangular form.
- The elimination operation at the k^{th} step is

$$\tilde{a}_{ij} = \tilde{a}_{ij} - (\tilde{a}_{ik}/\tilde{a}_{kk})\tilde{a}_{kj}, \quad i > k, \quad j \geq k$$

- Elimination requires three nested loops.
- The result of the elimination phase is represented by the image below.

$$\begin{bmatrix} x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \\ x & x & x & x & x \end{bmatrix} \longrightarrow \begin{bmatrix} x & x & x & x & x \\ 0 & x' & x' & x' & x' \\ 0 & 0 & x'' & x'' & x'' \\ 0 & 0 & 0 & x''' & x''' \end{bmatrix}$$

Gaussian Elimination

Summary

- Transform a linear system into (upper) triangular form. i.e. transform lower triangular part to zero
- Transformation is done by taking linear combinations of rows
- Example: $a = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}$
- If $a_1 \neq 0$, then

$$\begin{bmatrix} 1 & 0 \\ -a_2/a_1 & 1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} a_1 \\ 0 \end{bmatrix}$$

Gaussian Elimination Algorithm

Listing 2: Forward Elimination beta

```
1  given  $A, b$ 
2
3  for  $k = 1 \dots n - 1$ 
4      for  $i = k + 1 \dots n$ 
5          for  $j = k \dots n$ 
6               $a_{ij} = a_{ij} - (a_{ik}/a_{kk})a_{kj}$ 
7          end
8       $b_i = b_i - (a_{ik}/a_{kk})b_k$ 
9  end
10 end
```

- the multiplier can be moved outside the j -loop
- no reason to actually compute 0

Challenge: The loops over i and j may be exchanged—why would one be preferable?

Gaussian Elimination Algorithm

Listing 3: Forward Elimination

```
1  given  $A, b$ 
2
3  for  $k = 1 \dots n - 1$ 
4      for  $i = k + 1 \dots n$ 
5           $xmult = a_{ik}/a_{kk}$ 
6           $a_{ik} = xmult$ 
7          for  $j = k + 1 \dots n$ 
8               $a_{ij} = a_{ij} - (xmult)a_{kj}$ 
9          end
10          $b_i = b_i - (xmult)b_k$ 
11     end
12 end
```